

1.简介

浩辰CAD GRXSDK是浩辰结合GRX和Python开发的一组编程接口， 用于支持用户使用Python拓展GstarCAD平台软件的功能，例如创建自定义命令，创建自定义图元和实体，访问和修改图纸的各类图形数据，实现自定义绘图和图纸分析算法等。

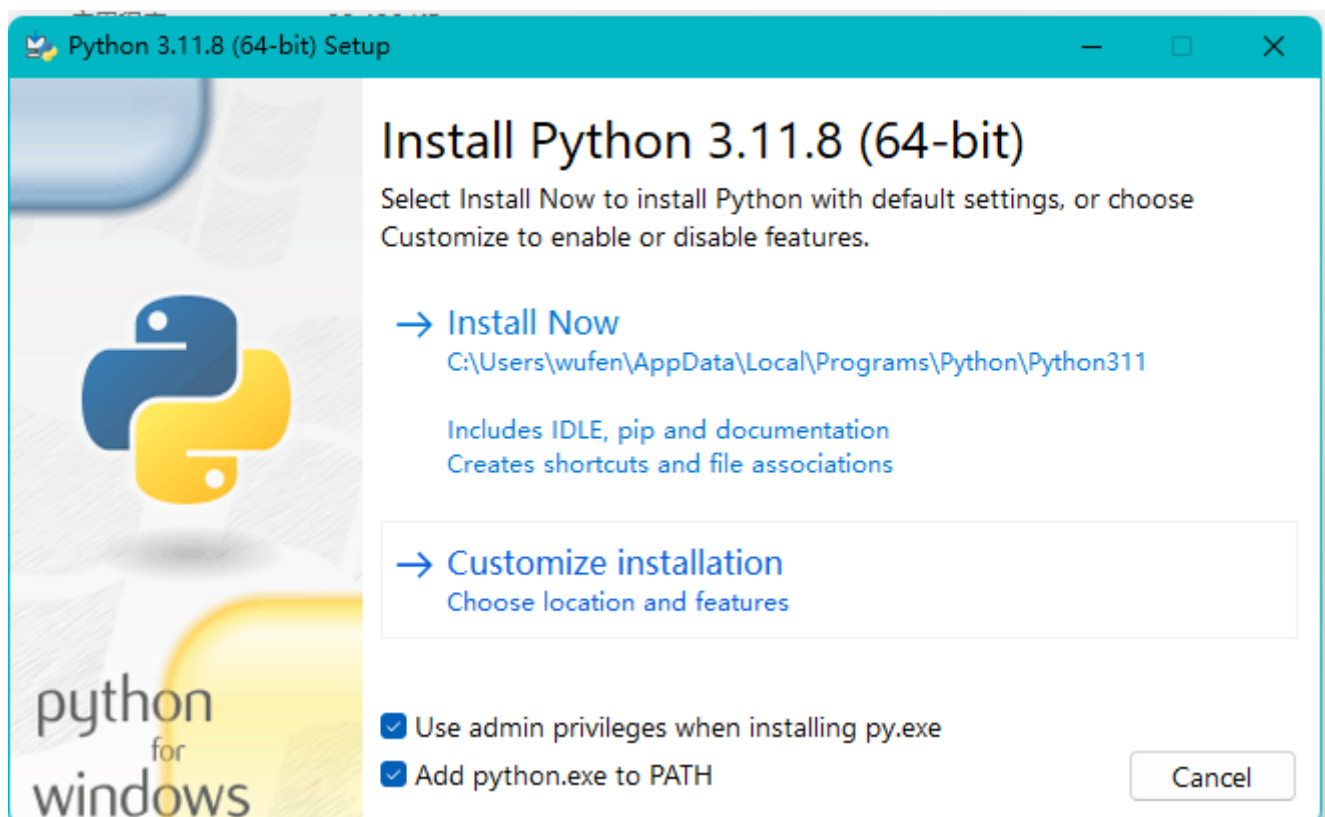
2.系统要求

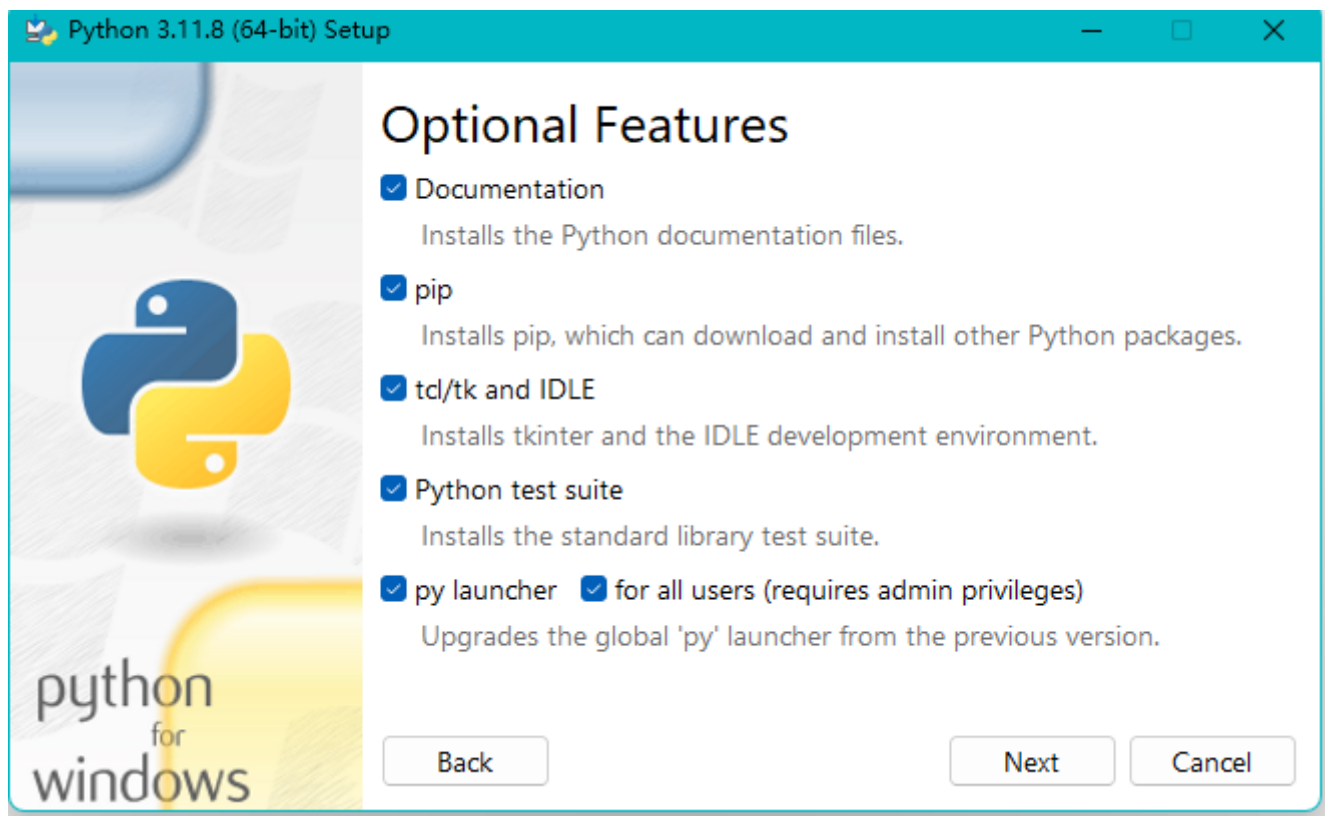
- Python3.11.8
- Windows10及以上操作系统

3.Python安装.

下载安装包: python-3.11.x-amd64.exe

管理员权限安装,保持默认安装选项。





安装完毕即可。

如果一切安装正常,打开cmd控制台,输入python命令,会有如下输出:

```
C:\Windows\system32\cmd.exe - python
Microsoft Windows [版本 10.0.22621.675]
(c) Microsoft Corporation。保留所有权利。

C:\Users\wufen>python
Python 3.11.8 (tags/v3.11.8:db85d51, Feb  6 2024, 22:03:32) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> _
```

如果不是,或者打印的python版本号不是我们安装的版本,请返回排查错误项.

4. 开发调试环境配置

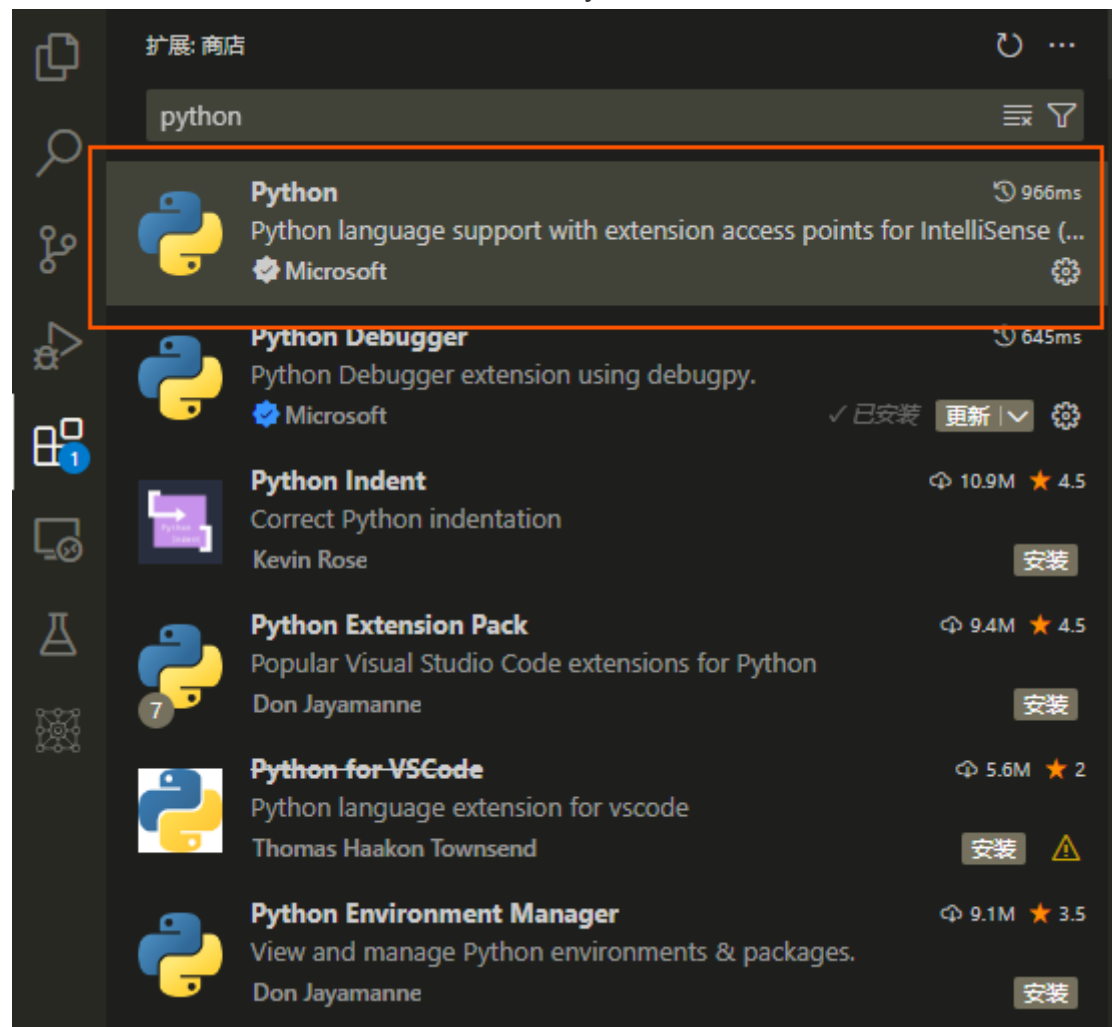
目前pygcad库是和gstarcad版本一起绑定发布的.没有pip安装的方式.

pygcad模块运行加载路径是随gstarcad启动自动识别的,不需要独立配置,但是pygcad模块不能被第三方python开发IDE环境识别,需要手动指定路径,手动指定路径后,即可在IDE中自动智能识别,自动补全pygcad接口.

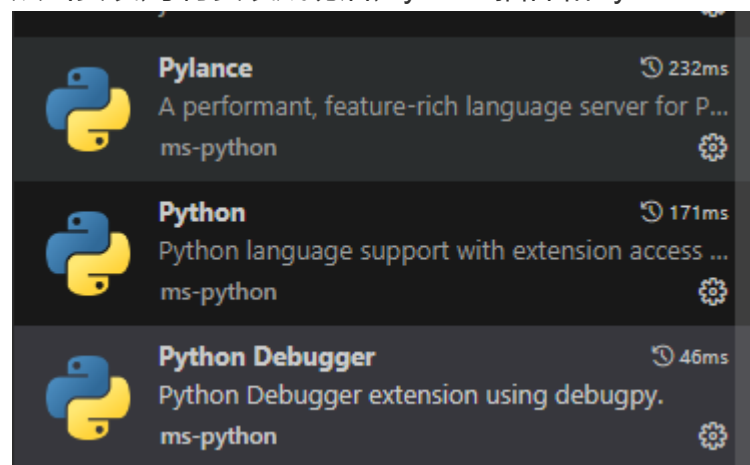
这里以VisualStudio Code(1.93.1 x86_64)为示例,介绍pygcad模块在IDE环境中的路径配置.其他IDE环境,比如(Pycharm,SublimeText)可自行同理配置.

4.1 安装python插件

在VisualStudio Code插件管理器中搜索Python,选择微软的插件安装,如下:

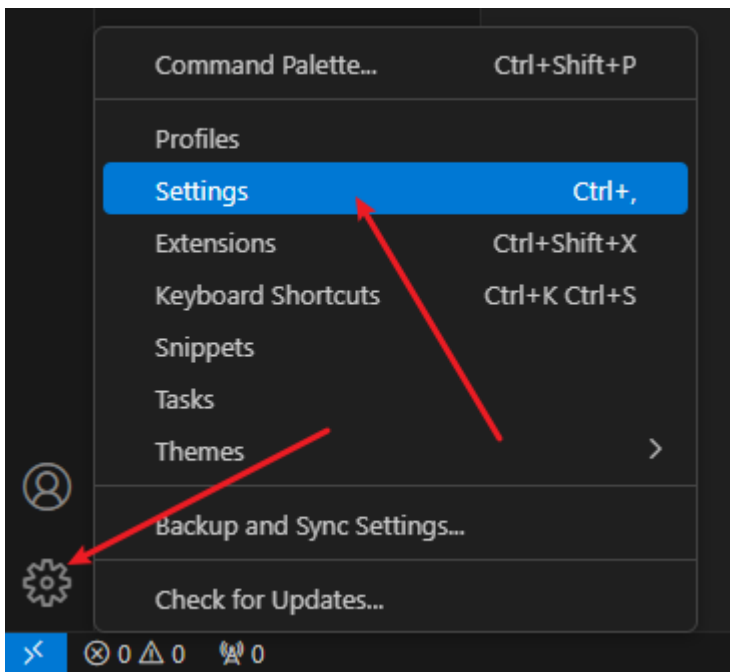


点击安装,等待安装成功后,Pylance插件和Python Debugger插件会同时安装.如下:

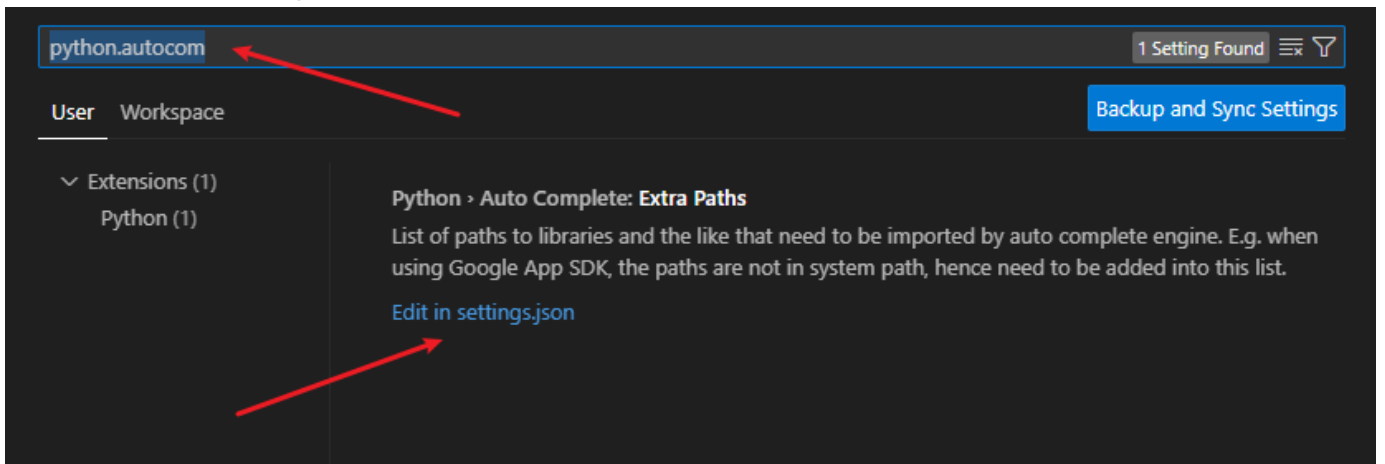


4.2 设置pygcad路径

1. 打开VisualStudio Code,打开一个工作目录.
2. 点击左下角齿轮图标,选择Settings.打开用户设置.

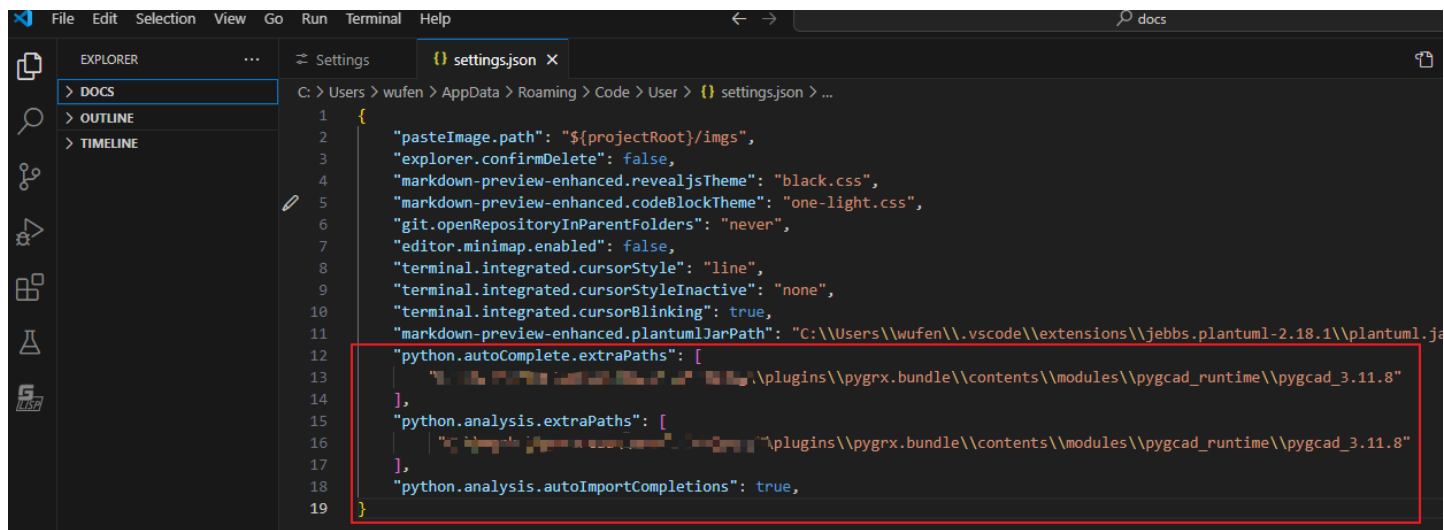


3. 在用户设置中搜索: Python.autocom ,结果为 Auto Complete:Extra Paths



4. 点击 Edit in Settings.json ,文件中的extraPaths中添加路径,此路径可能不同,请根据自己的安装位置相应修改.保存后,重启VisualStudio Code即可,完成配置.

```
"python.autoComplete.extraPaths": [  
    "C:\\Program Files\\浩辰软件\\浩辰CAD2025\\plugins\\pygrx.bundle\\contents\\modu  
],  
"python.analysis.extraPaths": [  
    "C:\\Program Files\\浩辰软件\\浩辰CAD2025\\plugins\\pygrx.bundle\\contents\\mod  
],  
"python.analysis.autoImportCompletions": true,
```



5. 在python文件中输入pygrx模块,可以自动提示模块中的方法和参数.具体参数类型的详细信息,可参考PygrxRef.chm帮助文档.

5. 代码示例

5.1 简单示例

以下python源码文件: myline.py 被gstarcad加载运行后,会自动注册 pyDrawLine 命令.在gstarcad中输入 pyDrawLine ,会在文档中绘制一条起点为 (0,0,0) ,终点为 (100,100,0) 的直线实体.

[myline.py](#)

```

#加载pygcad模块
from pygcad.core.runtime import *
from pygcad.pygrx import *

#@command 关键字会自动识别为gstaracad的自定义命令
@command()
def pyDrawLine():
    try:
        #获取当前数据库
        database = gcdbWorkingDatabase()

        #打开块表
        (status, blockTbl) = database.getBlockTable(GcDb.OpenMode.kForRead)
        #打开块表记录
        (status, record) = blockTbl.getAt(GCDB_MODEL_SPACE, GcDb.OpenMode.kForWrite)

        #关闭块表
        blockTbl.close()

        #新建直线实体
        line = GcDbLine(GcGePoint3d(0, 0, 0), GcGePoint3d(100, 100, 0))

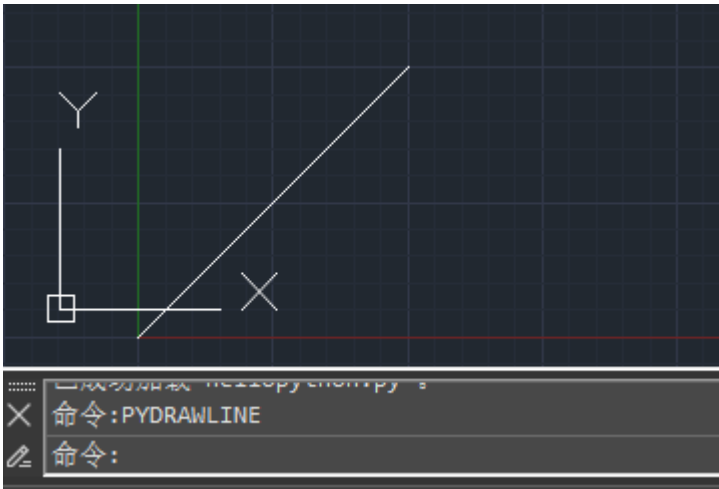
        #将直线实体添加到块表记录中
        (status, objId) = record.appendGcDbEntity(line)

        #关闭块表记录
        record.close()
        line.close()
    except Exception as err:
        #异常输出到gstaracad
        gcedPrompt('---- [ERROR]: %s'%err)

```

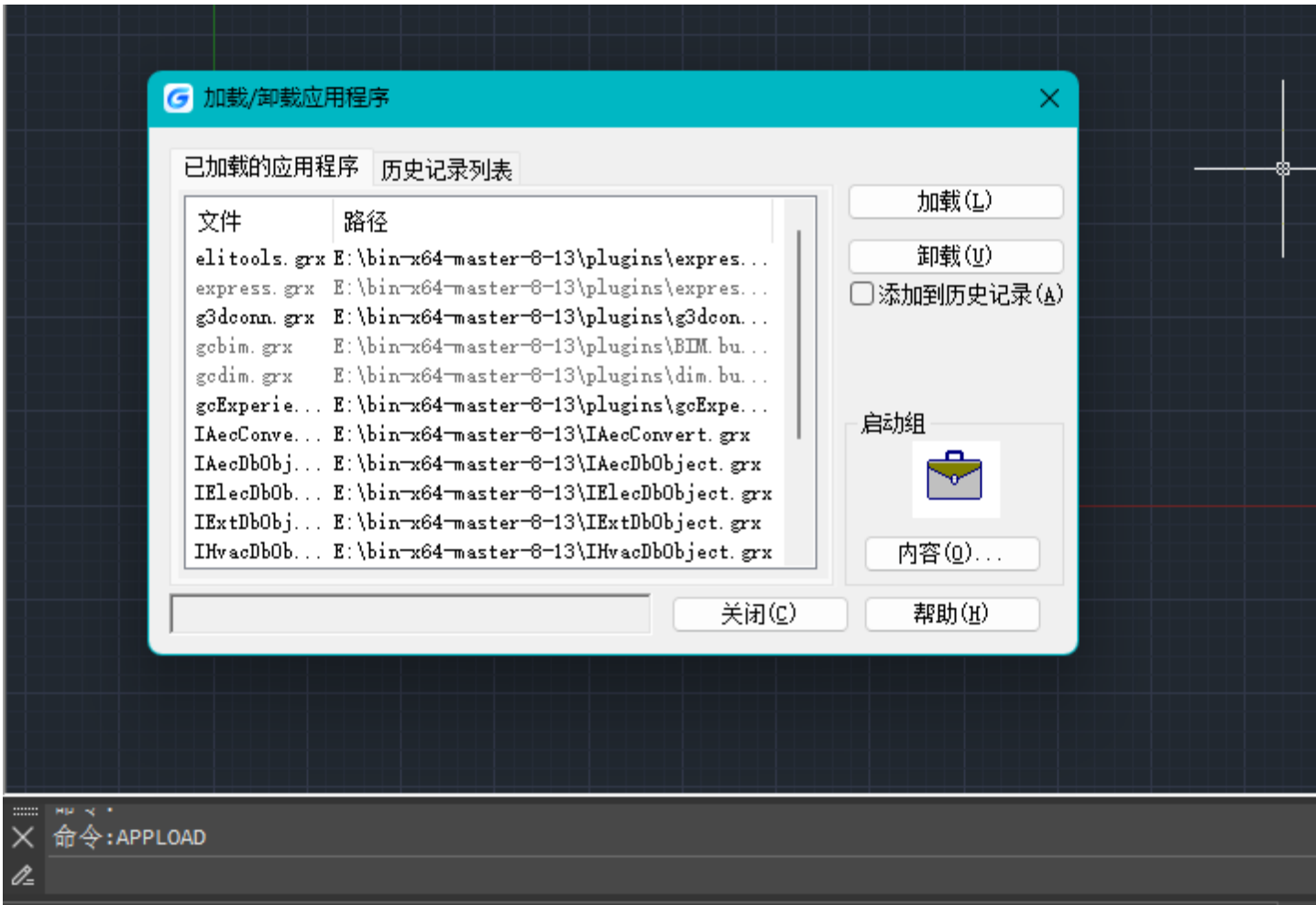
pygcad.core中包含了识别@command修饰符等辅助方法;
pygcad.pygrx中包含所有Grx接口方法;
@command 关键字会自动识别为gstaracad的自定义命令

效果如下:

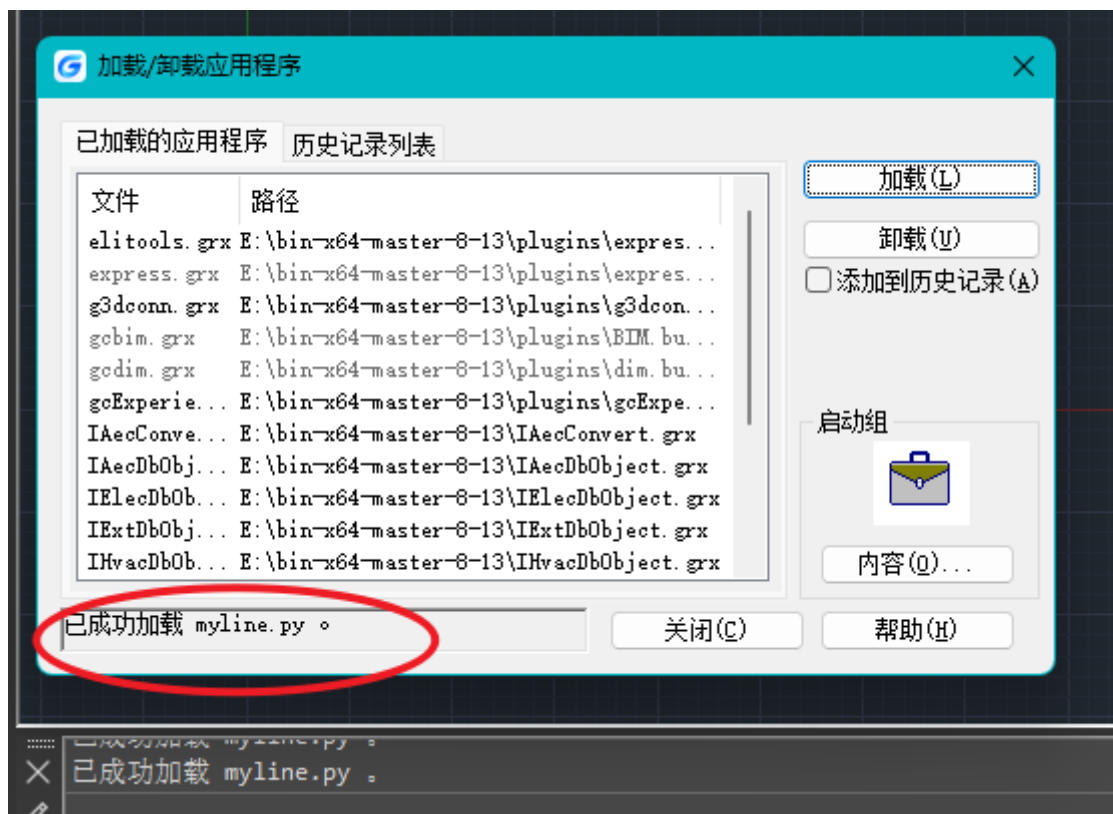


5.2 怎样加载简单示例

打开浩辰CAD软件，在命令框中输入apload命令，打开插件加载对话框，如下：



然后点击加载按钮,在弹出的文件查找对话框中,选择myline.py文件,点击打开.
myline.py文件即可加载.结果如下:



点击关闭按钮,即可在gstarcad命令框中识别输入的 pyDrawLine 命令.

5.3 修改自定义命令名

函数pyDrawLine()被@command()修饰。即可自动注册为GCAD命令 PYDRAWLINE 。
如果不想使用函数名作为命令, 可以在装饰器函数command()中修改,例如:

```
@command(local_name='PY_MY_CMD')
def pyDrawLine():
    print("自定义命令为PY_MY_CMD")
    ...
```

此时myline.py被加载后,在命令框中应输入 py_my_cmd ,绘制一条直线.

@command 是一个装饰器函数,定义如下:

```
def command(local_name="", global_name="", group_name="", cmd_flags=0)
```

5.4 拖拽椭圆命令示例

一个自定义命令 PyEllipse ,通过鼠标指定长轴和短轴,拖拽绘制一个椭圆实体.源码如下:

[elipsjig.py](#)


```

"""
..Description:
    提供一个Jig接口实现椭圆的绘制
"""

from pygcad.core.runtime import *
from pygcad.pygrx import *
import traceback

class EllipseJig(GcEdJig):
    def __init__(self, center: GcGePoint3d, normal: GcGeVector3d):
        GcEdJig.__init__(self)

        self._ellipse = GcDbEllipse()
        self._center_pt = center
        self._normal = normal
        self._radius_ratio = 0.0001
        self._prompt_counter = 0
        self._axis_pt = GcGePoint3d()

        self.tmp_pt = GcGePoint3d()
        self.tmp_ratio = 0.0001

        rb = resbuf()
        gcedGetVar("VIEWSIZE", rb)
        major_axis_init_offset = rb.resval.rreal / 1000.0
        self._major_axis = GcGeVector3d(major_axis_init_offset, 0, 0)

    def sampler(self):
        status = GcEdJig.kNormal

        input_flags = GcEdJig.UserInputControls.kAccept3dCoordinates \
            | GcEdJig.UserInputControls.kNoNegativeResponseAccepted \
            | GcEdJig.UserInputControls.kNoZeroResponseAccepted
        self.setUserInputControls(GcEdJig.UserInputControls(input_flags))

        if self._prompt_counter == 0:
            status = self.acquirePoint(self._axis_pt, self._center_pt)
            if not self._axis_pt.isEqualTo(self.tmp_pt):
                self.tmp_pt = GcGePoint3d(self._axis_pt.x, self._axis_pt.y, self._axis_pt.z)
            elif status != GcEdJig.kNormal:
                return GcEdJig.kNoChange

        elif self._prompt_counter == 1:
            status, self._radius_ratio = self.acquireDist(self._center_pt)

```

```

        if self.tmp_ratio != self._radius_ratio:
            self.tmp_ratio = self._radius_ratio
        elif status != GcEdJig.kNormal:
            return GcEdJig.kNoChange

    return status

def update(self):
    if self._prompt_counter == 0:
        self._major_axis = self._axis_pt - self._center_pt
    elif self._prompt_counter == 1:
        self._radius_ratio = self._radius_ratio / self._major_axis.length()

    self._ellipse.set(self._center_pt, self._normal, self._major_axis, self._radius_ratio)
    return True

def entity(self):
    return self._ellipse

def doIt(self):
    self._ellipse.set(self._center_pt, self._normal, self._major_axis, self._radius_ratio)
    self.setDispPrompt("\nEllipse major axis: ")
    status = self.drag()

    self._prompt_counter += 1
    self.setDispPrompt("\nEllipse minor axis: ")
    status = self.drag()
    self.append(self._ellipse)

def createEllipse():
    tmp_pt = GcGePoint3d()
    gcedGetPoint(None, "\nEllipse center point: ", tmp_pt)

    rb_from = resbuf()
    rb_to = resbuf()
    rb_from.restype = RTSHORT
    rb_from.resval.rint = 1
    rb_to.restype = RTSHORT
    rb_to.resval.rint = 0
    gcedTrans(tmp_pt, rb_from, rb_to, False, tmp_pt)

    x = gcdbWorkingDatabase().ucsxdir()
    y = gcdbWorkingDatabase().ucsydir()
    normal = x.crossProduct(y)
    normal.normalize()

```

```
jig = EllipseJig(tmp_pt, normal)
jig.doIt()
```

```
@command()
```

```
def PyEllipse():
```

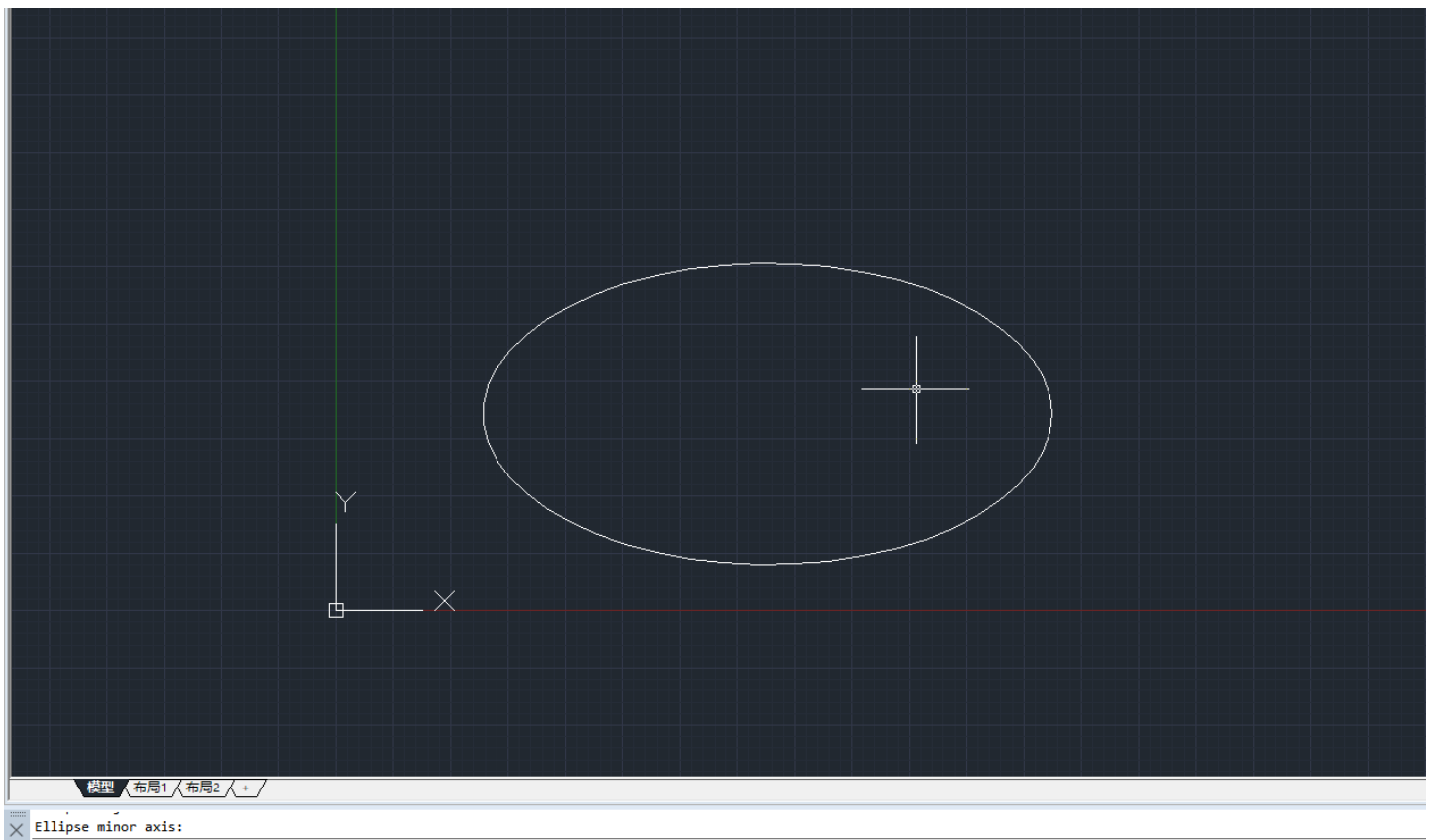
```
    try:
```

```
        createEllipse()
```

```
    except Exception as err:
```

```
        gcutPrintf(traceback.format_exc())
```

效果如下:



6. 常见问题

1. python文件加载失败.

```
命令:APPLOAD
加载失败, 请在应用管理器中打开"Python二次开发接口"选项 无法加载 linejig.py 文件。
```

或者

```
Failed to load, please enable the "Python Development Interface" in the application manager.Unable to load linejig.py file.
```

解决方法:

菜单选择, 应用软件 --> 应用管理器 --> python二次开发接口 ,点击开启.

2. python接口模块初始化失败.

```
命令:Python接口模块加载异常,请确认已安装Python,且添加到Path环境变量!PyVersion:pyloader3_8_10.dll code:126
```

或者

```
Command: Python interface module loaded abnormal! Please insure installed corresponding version Python,and add path to system environment variable!PyVersion:pyloader3_8_10.dll code:126
```

解决方法:

在windows系统环境变量的Path路径下,添加python3.11.x的安装路径,重启GstarCAD.